

Spotify Popularity: Exploratory Data Analysis

Jacoby Ramsey

2026-08-23

Contents

Purpose	2
Load Packages	2
Visual Theme	2
Load Data	3
Data Structure	3
Missing Values	4
Duplicate Track IDs	6
Popularity Distribution	7
Binary Popularity Classification	8
Genre Composition	9
Popularity by Genre	11
Audio Feature Summary	13
Audio Feature Distributions	14
Audio Characteristics: Popular vs. Not Popular	15
Selected Feature Relationships with Popularity	17
Correlation Analysis	20
Genre-Level Audio Profiles	22

Purpose

This report focuses only on exploratory data analysis (EDA). The goal is to understand the structure of the Spotify dataset, examine the distribution of popularity, identify differences across genres, and explore how measurable audio characteristics relate to popularity before any predictive modeling is performed.

The analysis uses the complete dataset for descriptive and exploratory work. The 20% modeling sample used in the predictive-modeling report is not used here.

Load Packages

```
library(dplyr)
library(tidyr)
library(ggplot2)
library(janitor)
library(scales)
library(corrplot)
```

Visual Theme

```
theme_spotify <- function() {
  theme_minimal(base_size = 12) +
    theme(
      plot.background = element_rect(fill = "#121212", color = NA),
      panel.background = element_rect(fill = "#181818", color = NA),
      panel.border = element_blank(),
      panel.grid.major = element_line(color = "#2A2A2A", linewidth = 0.4),
      panel.grid.minor = element_blank(),
      axis.text = element_text(color = "#B3B3B3"),
      axis.title = element_text(color = "#F5F5F5", face = "bold"),
      plot.title = element_text(color = "#1DB954", face = "bold", size = 16),
      plot.subtitle = element_text(color = "#D9D9D9", size = 11),
      plot.caption = element_text(color = "#B3B3B3", size = 9),
      legend.background = element_rect(fill = "#181818", color = NA),
      legend.key = element_rect(fill = "#181818", color = NA),
      legend.text = element_text(color = "#F5F5F5"),
      legend.title = element_text(color = "#1DB954", face = "bold"),
      strip.background = element_rect(fill = "#1DB954", color = NA),
      strip.text = element_text(color = "#121212", face = "bold")
    )
}
```

Load Data

```
spotify <- read.csv("SpotifyFeatures.csv", stringsAsFactors = TRUE)
```

```
nrow(spotify)
```

```
## [1] 232725
```

```
ncol(spotify)
```

```
## [1] 18
```

The dataset contains 232,725 observations and 18 original variables.

Data Structure

```
str(spotify)
```

```
## 'data.frame':    232725 obs. of  18 variables:
## $ genre          : Factor w/ 27 levels "A Capella","Alternative",...: 16 16 16 16 16 16 16 16 16 16 ...
## $ artist_name    : Factor w/ 14564 levels "!!!","¡MAYDAY!",...: 5255 8326 6544 5255 4112 5255 8326 ...
## $ track_name     : Factor w/ 148615 levels "____45____",...: 18615 93903 32611 31339 91776 69420 9 ...
## $ track_id       : Factor w/ 176774 levels "00021Wy6AyMbLP2tqiJ86e",...: 4971 4768 5608 8233 10160 ...
## $ popularity     : int  0 1 3 0 4 0 2 15 0 10 ...
## $ acousticness   : num  0.611 0.246 0.952 0.703 0.95 0.749 0.344 0.939 0.00104 0.319 ...
## $ danceability   : num  0.389 0.59 0.663 0.24 0.331 0.578 0.703 0.416 0.734 0.598 ...
## $ duration_ms    : int  99373 137373 170267 152427 82625 160627 212293 240067 226200 152694 ...
## $ energy         : num  0.91 0.737 0.131 0.326 0.225 0.0948 0.27 0.269 0.481 0.705 ...
## $ instrumentalness: num  0 0 0 0 0.123 0 0 0 0.00086 0.00125 ...
## $ key            : Factor w/ 12 levels "A","A#","B","C",...: 5 10 4 5 9 5 5 10 4 11 ...
## $ liveness       : num  0.346 0.151 0.103 0.0985 0.202 0.107 0.105 0.113 0.0765 0.349 ...
## $ loudness       : num  -1.83 -5.56 -13.88 -12.18 -21.15 ...
## $ mode           : Factor w/ 2 levels "Major","Minor": 1 2 2 1 1 1 1 1 1 1 ...
## $ speechiness    : num  0.0525 0.0868 0.0362 0.0395 0.0456 0.143 0.953 0.0286 0.046 0.0281 ...
## $ tempo          : num  167 174 99.5 171.8 140.6 ...
## $ time_signature : Factor w/ 5 levels "0/4","1/4","3/4",...: 4 4 5 4 4 4 4 4 4 4 ...
## $ valence        : num  0.814 0.816 0.368 0.227 0.39 0.358 0.533 0.274 0.765 0.718 ...
```

```
summary(spotify)
```

##	genre	artist_name	track_name
##	Comedy : 9681	Giuseppe Verdi : 1394	Home : 100
##	Soundtrack: 9646	Giacomo Puccini : 1137	You : 71
##	Indie : 9543	Kimbo Children's Music : 971	Intro : 69
##	Jazz : 9441	Nobuo Uematsu : 825	Stay : 63
##	Pop : 9386	Richard Wagner : 804	Wake Up: 59
##	Electronic: 9377	Wolfgang Amadeus Mozart: 800	Closer : 58

```
## (Other) :175651 (Other) :226794 (Other):232305
## track_id popularity acousticness
## OUEORhnRaEYsiYgXpyLoZc: 8 Min. : 0.00 Min. :0.0000
## 0wY9rA9fJkuESyYm9uzVK5: 8 1st Qu.: 29.00 1st Qu.:0.0376
## 3R73Y7X53MIQZWnKloWq5i: 8 Median : 43.00 Median :0.2320
## 3uSSjnDMmoyERaAK9KvpJR: 8 Mean : 41.13 Mean :0.3686
## 6Alte2Iej1QKlaofpJczW1: 8 3rd Qu.: 55.00 3rd Qu.:0.7220
## 6sVQNUvcVFTXvlk3ec0ngd: 8 Max. :100.00 Max. :0.9960
## (Other) :232677
## danceability duration_ms energy instrumentalness
## Min. :0.0569 Min. : 15387 Min. :2.03e-05 Min. :0.0000000
## 1st Qu.:0.4350 1st Qu.: 182857 1st Qu.:3.85e-01 1st Qu.:0.0000000
## Median :0.5710 Median : 220427 Median :6.05e-01 Median :0.0000443
## Mean :0.5544 Mean : 235122 Mean :5.71e-01 Mean :0.1483012
## 3rd Qu.:0.6920 3rd Qu.: 265768 3rd Qu.:7.87e-01 3rd Qu.:0.0358000
## Max. :0.9890 Max. :5552917 Max. :9.99e-01 Max. :0.9990000
##
## key liveness loudness mode
## C :27583 Min. :0.00967 Min. : -52.457 Major:151744
## G :26390 1st Qu.:0.09740 1st Qu.: -11.771 Minor: 80981
## D :24077 Median :0.12800 Median : -7.762
## C# :23201 Mean :0.21501 Mean : -9.570
## A :22671 3rd Qu.:0.26400 3rd Qu.: -5.501
## F :20279 Max. :1.00000 Max. : 3.744
## (Other):88524
## speechiness tempo time_signature valence
## Min. :0.0222 Min. : 30.38 0/4: 8 Min. :0.0000
## 1st Qu.:0.0367 1st Qu.: 92.96 1/4: 2608 1st Qu.:0.2370
## Median :0.0501 Median :115.78 3/4: 24111 Median :0.4440
## Mean :0.1208 Mean :117.67 4/4:200760 Mean :0.4549
## 3rd Qu.:0.1050 3rd Qu.:139.05 5/4: 5238 3rd Qu.:0.6600
## Max. :0.9670 Max. :242.90 Max. :1.0000
##
```

This output provides the data type of each variable and basic descriptive statistics. The dataset includes song identifiers, genre information, popularity scores, and multiple continuous audio characteristics.

Missing Values

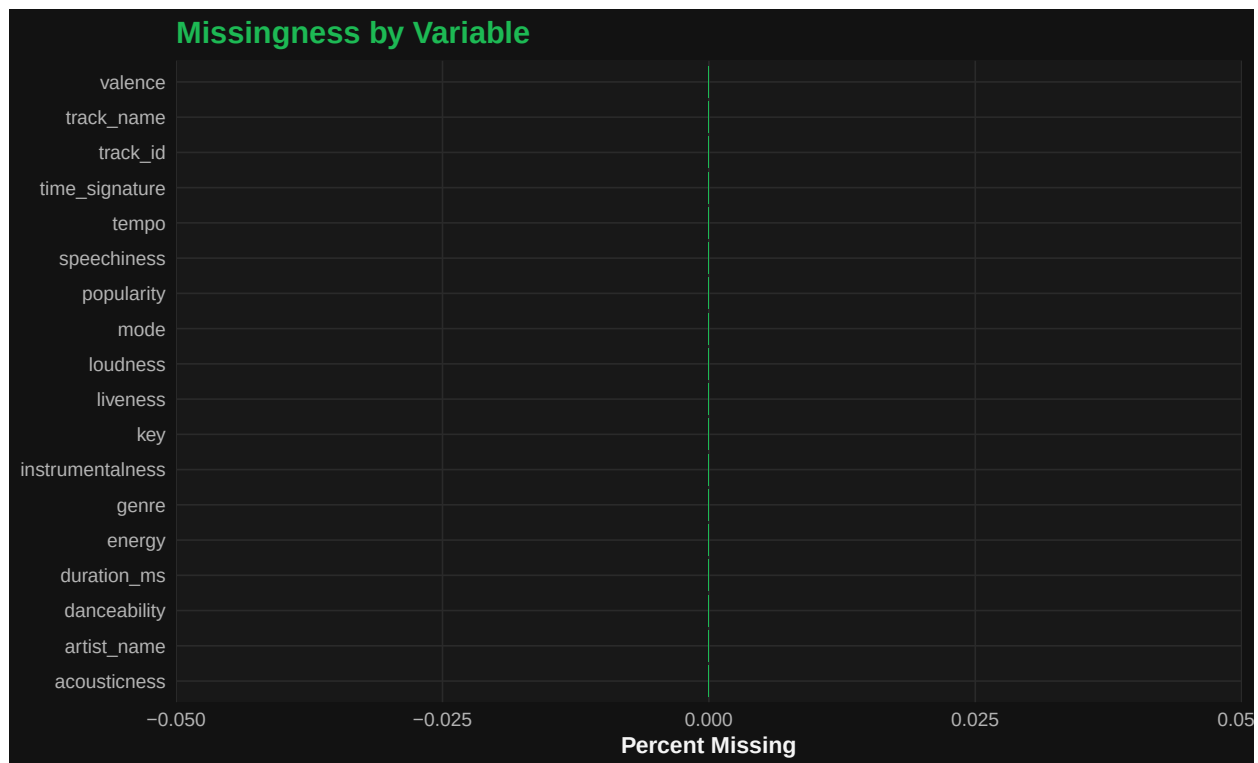
```
missing_summary <- data.frame(
  variable = names(spotify),
  missing = colSums(is.na(spotify))
) %>%
  mutate(percent_missing = 100 * missing / nrow(spotify)) %>%
  arrange(desc(missing))

knitr::kable(missing_summary, digits = 2, caption = "Missing Values by Variable")
```

Table 1: Missing Values by Variable

	variable	missing	percent_missing
genre	genre	0	0
artist_name	artist_name	0	0
track_name	track_name	0	0
track_id	track_id	0	0
popularity	popularity	0	0
acousticness	acousticness	0	0
danceability	danceability	0	0
duration_ms	duration_ms	0	0
energy	energy	0	0
instrumentalness	instrumentalness	0	0
key	key	0	0
liveness	liveness	0	0
loudness	loudness	0	0
mode	mode	0	0
speechiness	speechiness	0	0
tempo	tempo	0	0
time_signature	time_signature	0	0
valence	valence	0	0

```
ggplot(missing_summary, aes(x = reorder(variable, percent_missing), y = percent_missing)) +
  geom_col(fill = "#1DB954") +
  coord_flip() +
  labs(
    title = "Missingness by Variable",
    x = NULL,
    y = "Percent Missing"
  ) +
  theme_spotify()
```



The dataset contains no missing values if all percentages above are zero. This reduces the need for imputation and simplifies subsequent analysis.

Duplicate Track IDs

Because the same Spotify track can appear more than once in the dataset, it is useful to examine repeated track IDs before interpreting song-level counts.

```
duplicate_summary <- spotify %>%
  count(track_id, sort = TRUE) %>%
  summarise(
    unique_track_ids = n(),
    repeated_track_ids = sum(n > 1),
    maximum_repetitions = max(n)
  )

knitr::kable(duplicate_summary, caption = "Track ID Duplication Summary")
```

Table 2: Track ID Duplication Summary

unique_track_ids	repeated_track_ids	maximum_repetitions
176774	35124	8

```
top_repeated <- spotify %>%
  count(track_id, track_name, artist_name, sort = TRUE) %>%
  slice_head(n = 10)
```

```
knitr::kable(top_repeated, caption = "Most Frequently Repeated Tracks")
```

Table 3: Most Frequently Repeated Tracks

track_id	track_name	artist_name	n
0UE0RhnRaEYsiYgXpyLoZc	Hallucinations	dvsn	8
0wY9rA9fJkuESyYm9uzVK5	D'Evils	SiR	8
3R73Y7X53MIQZWnKloWq5i	Monte Carlo (feat. WET)	Toro y Moi	8
3uSSjnDMmoyERaAK9KvpJR	Too Deep	dvsn	8
6AIte2Iej1QKlaofpjCzW1	With Me	dvsn	8
6sVQNUvcVFTXvlk3ec0ngd	Cherry-coloured Funk	Cocteau Twins	8
03wKMRNYVvw6s9nm4I4jUS	Wolf Like Me	TV On The Radio	7
0725YWm6Z0TpZ6wrNk64Eb	Super Rich Kids	Frank Ocean	7
0aersVHlGGXcHeRvHtuaSt	Dancing On Glass	St. Lucia	7
0gKiONJeEtwb3hps4sUgyn	Changes	Langhorne Slim	7

This check helps determine whether repeated songs could influence descriptive results. Repeated entries are not automatically errors; they may reflect how the source dataset was assembled or how tracks are represented across categories.

Popularity Distribution

```
median_popularity <- median(spotify$popularity, na.rm = TRUE)

popularity_summary <- spotify %>%
  summarise(
    minimum = min(popularity),
    q1 = quantile(popularity, 0.25),
    median = median(popularity),
    mean = mean(popularity),
    q3 = quantile(popularity, 0.75),
    maximum = max(popularity),
    sd = sd(popularity)
  )

knitr::kable(popularity_summary, digits = 2, caption = "Popularity Summary Statistics")
```

Table 4: Popularity Summary Statistics

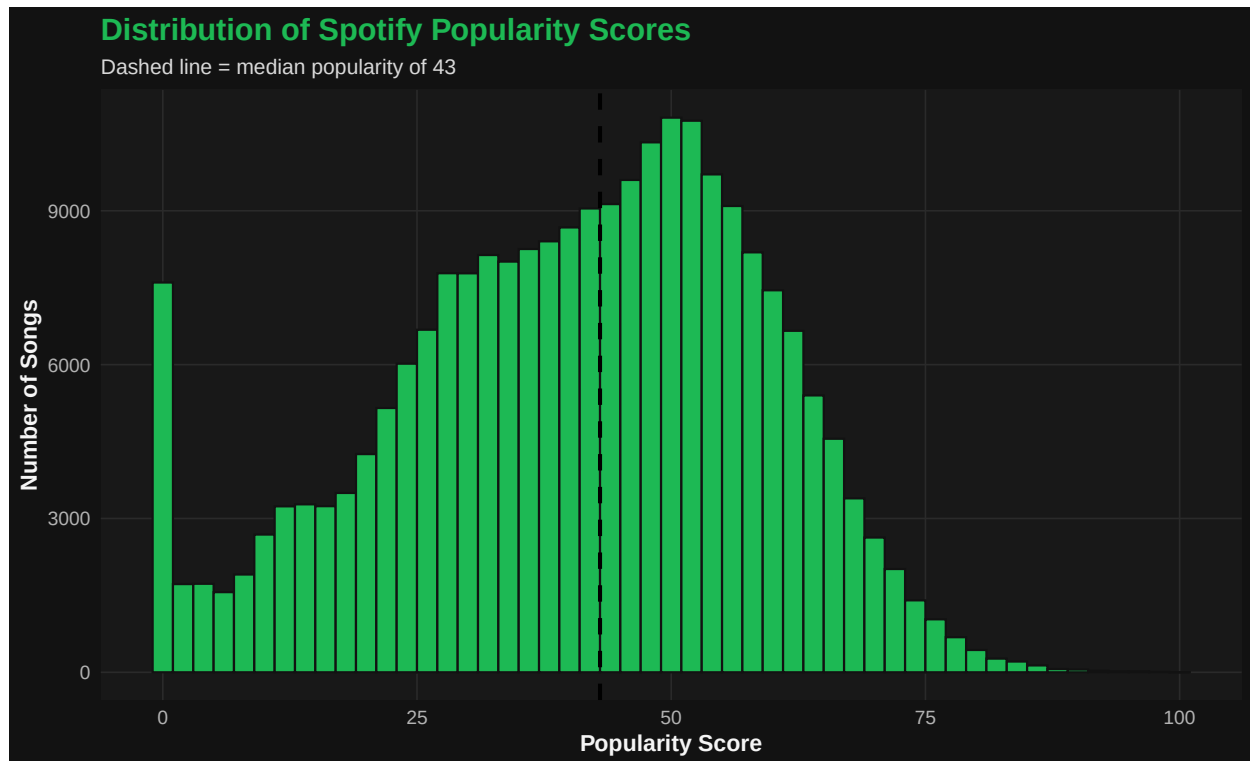
minimum	q1	median	mean	q3	maximum	sd
0	29	43	41.13	55	100	18.19

```
ggplot(spotify, aes(x = popularity)) +
  geom_histogram(binwidth = 2, fill = "#1DB954", color = "#121212") +
  geom_vline(xintercept = median_popularity, linetype = "dashed", linewidth = 1) +
  labs(
```

```

title = "Distribution of Spotify Popularity Scores",
subtitle = paste("Dashed line = median popularity of", median_popularity),
x = "Popularity Score",
y = "Number of Songs"
) +
theme_spotify()

```



The popularity distribution shows how songs are spread across Spotify's popularity scale. The median is also used to create the binary popularity outcome used in the predictive-modeling analysis.

Binary Popularity Classification

```

spotify_eda <- spotify %>%
  mutate(
    popular_binary = if_else(popularity >= median_popularity, 1L, 0L),
    popularity_group = factor(
      popular_binary,
      levels = c(0, 1),
      labels = c("Not Popular", "Popular")
    )
  )

class_balance <- spotify_eda %>%
  count(popularity_group) %>%
  mutate(percent = 100 * n / sum(n))

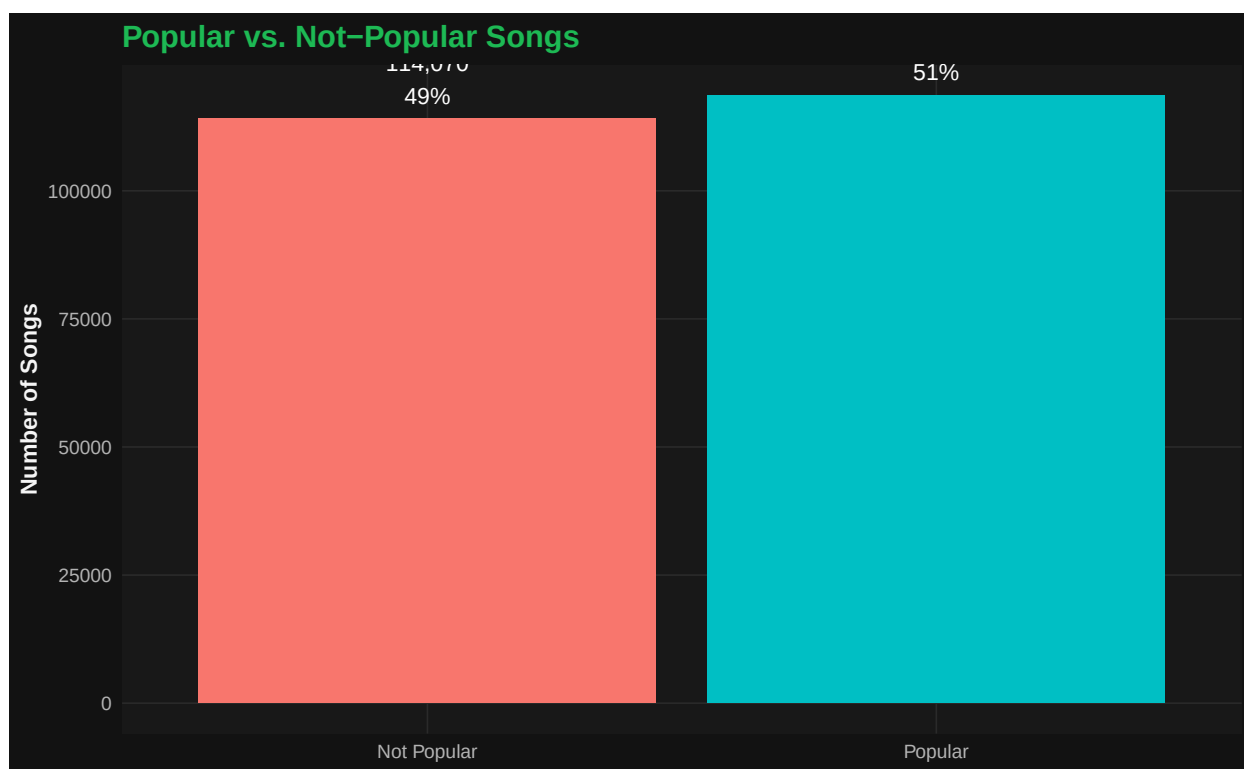
knitr::kable(class_balance, digits = 2, caption = "Popularity Class Balance")

```


Table 5: Popularity Class Balance

popularity_group	n	percent
Not Popular	114070	49.01
Popular	118655	50.99

```
ggplot(class_balance, aes(x = popularity_group, y = n, fill = popularity_group)) +
  geom_col(show.legend = FALSE) +
  geom_text(aes(label = paste0(comma(n), "\n", round(percent, 1), "%")),
            vjust = -0.3, color = "white") +
  labs(
    title = "Popular vs. Not-Popular Songs",
    x = NULL,
    y = "Number of Songs"
  ) +
  theme_spotify()
```



A nearly even class distribution is useful because later classification accuracy is less likely to be driven simply by one class being much larger than the other.

Genre Composition

```
genre_counts <- spotify %>%
  count(genre, sort = TRUE)

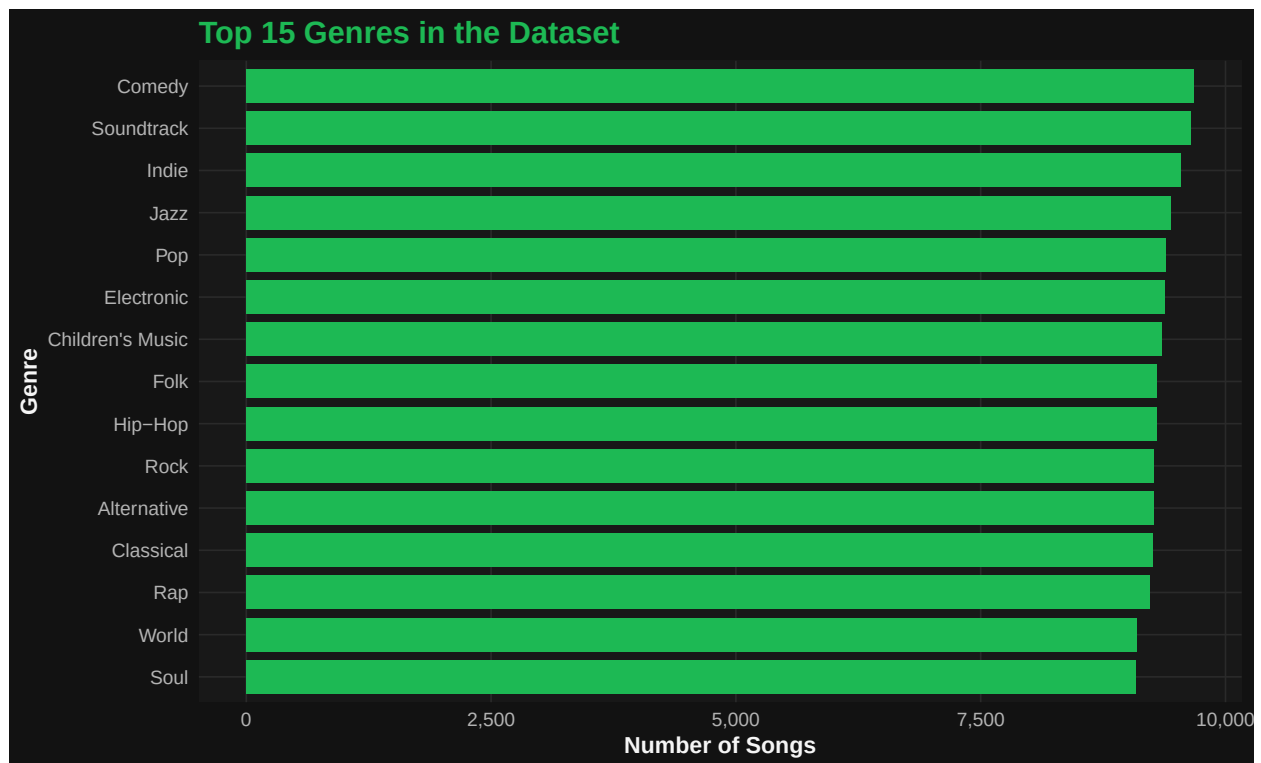
knitr::kable(head(genre_counts, 15), caption = "Top 15 Genres by Number of Songs")
```

Table 6: Top 15 Genres by Number of Songs

genre	n
Comedy	9681
Soundtrack	9646
Indie	9543
Jazz	9441
Pop	9386
Electronic	9377
Children's Music	9353
Folk	9299
Hip-Hop	9295
Rock	9272
Alternative	9263
Classical	9256
Rap	9232
World	9096
Soul	9089

```
top_genres <- genre_counts %>% slice_head(n = 15)

ggplot(top_genres, aes(x = reorder(genre, n), y = n)) +
  geom_col(fill = "#1DB954", width = 0.8) +
  coord_flip() +
  scale_y_continuous(labels = comma) +
  labs(
    title = "Top 15 Genres in the Dataset",
    x = "Genre",
    y = "Number of Songs"
  ) +
  theme_spotify()
```



The genre counts help determine whether the dataset is concentrated in only a few categories. Similar genre counts reduce the risk that the descriptive results are dominated by one genre alone.

Popularity by Genre

```
genre_popularity <- spotify %>%
  group_by(genre) %>%
  summarise(
    average_popularity = mean(popularity, na.rm = TRUE),
    median_popularity = median(popularity, na.rm = TRUE),
    song_count = n(),
    .groups = "drop"
  ) %>%
  arrange(desc(average_popularity))

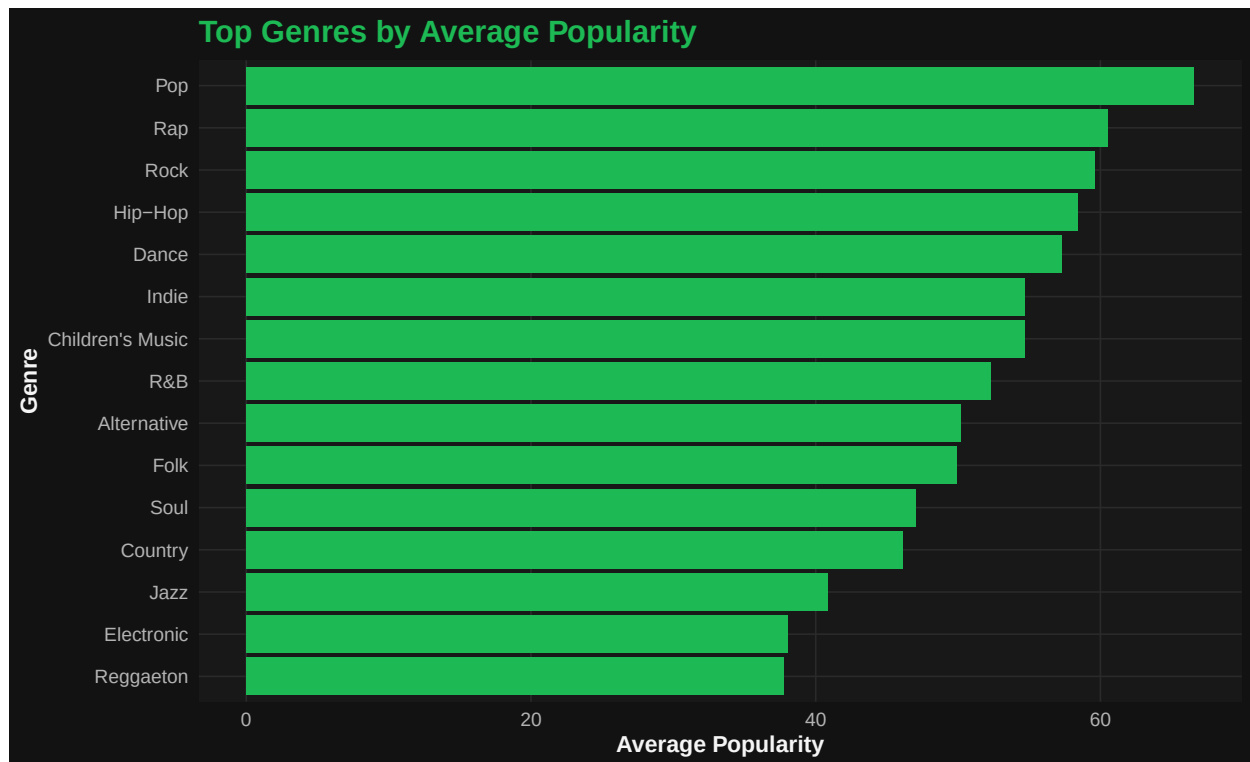
knitr::kable(head(genre_popularity, 15), digits = 2,
  caption = "Top Genres by Average Popularity")
```

Table 7: Top Genres by Average Popularity

genre	average_popularity	median_popularity	song_count
Pop	66.59	66	9386
Rap	60.53	59	9232
Rock	59.62	59	9272
Hip-Hop	58.42	57	9295

genre	average_popularity	median_popularity	song_count
Dance	57.28	57	8701
Indie	54.70	54	9543
Children's Music	54.66	54	9353
R&B	52.31	51	8992
Alternative	50.21	49	9263
Folk	49.94	49	9299
Soul	47.03	46	9089
Country	46.10	45	8664
Jazz	40.82	40	9441
Electronic	38.06	37	9377
Reggaeton	37.74	35	8927

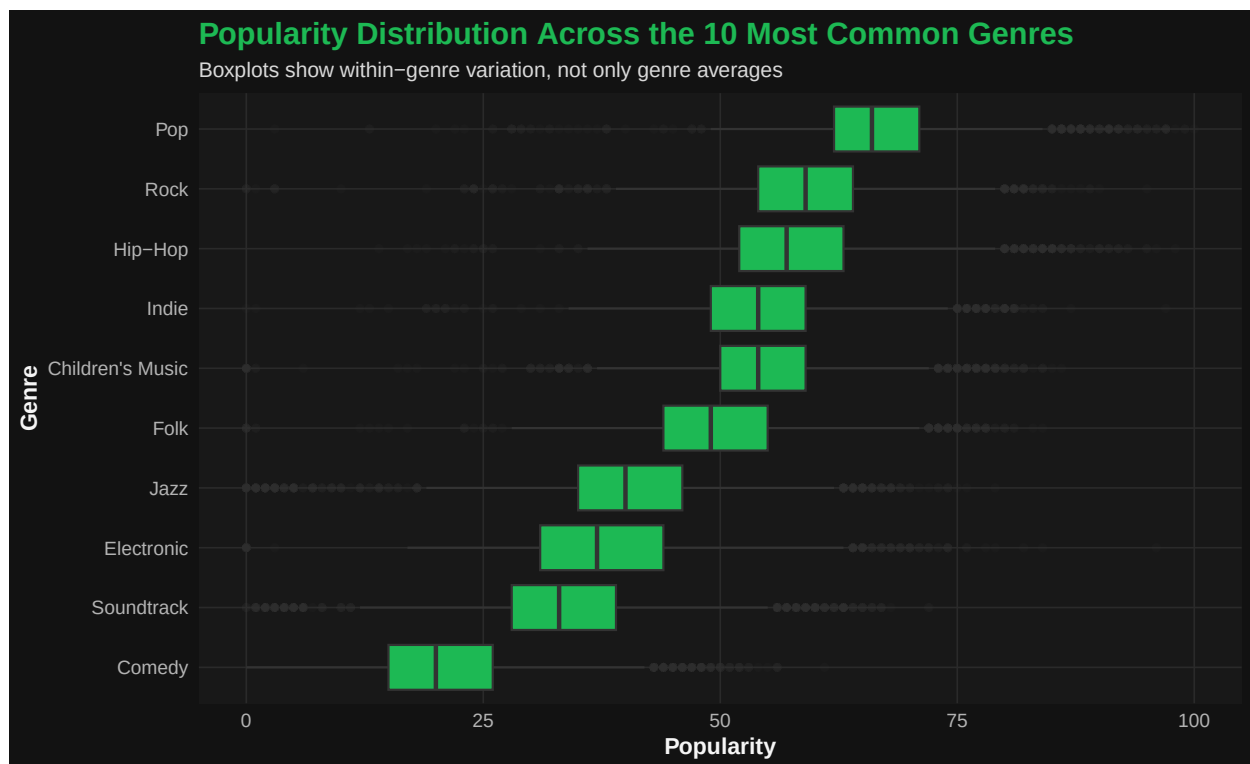
```
genre_popularity %>%
  slice_head(n = 15) %>%
  ggplot(aes(x = reorder(genre, average_popularity), y = average_popularity)) +
  geom_col(fill = "#1DB954") +
  coord_flip() +
  labs(
    title = "Top Genres by Average Popularity",
    x = "Genre",
    y = "Average Popularity"
  ) +
  theme_spotify()
```



```
top_10_genres <- genre_counts %>%
  slice_head(n = 10) %>%
```

```
pull(genre)

spotify %>%
  filter(genre %in% top_10_genres) %>%
  ggplot(aes(x = reorder(genre, popularity, median), y = popularity)) +
  geom_boxplot(fill = "#1DB954", outlier.alpha = 0.15) +
  coord_flip() +
  labs(
    title = "Popularity Distribution Across the 10 Most Common Genres",
    subtitle = "Boxplots show within-genre variation, not only genre averages",
    x = "Genre",
    y = "Popularity"
  ) +
  theme_spotify()
```



The bar chart compares average popularity, while the boxplots show the full distribution within major genres. This distinction is important because two genres can have similar averages while having very different spreads and medians.

Audio Feature Summary

```
audio_features <- c(
  "acousticness", "danceability", "duration_ms", "energy",
  "instrumentalness", "liveness", "loudness", "speechiness",
  "tempo", "valence"
)
```

```
feature_summary <- spotify %>%
  summarise(across(
    all_of(audio_features),
    list(
      mean = ~mean(.x, na.rm = TRUE),
      median = ~median(.x, na.rm = TRUE),
      sd = ~sd(.x, na.rm = TRUE)
    )
  )) %>%
  pivot_longer(
    everything(),
    names_to = c("feature", ".value"),
    names_pattern = "(.+)_(mean|median|sd)$"
  )

knitr::kable(feature_summary, digits = 3, caption = "Audio Feature Summary Statistics")
```

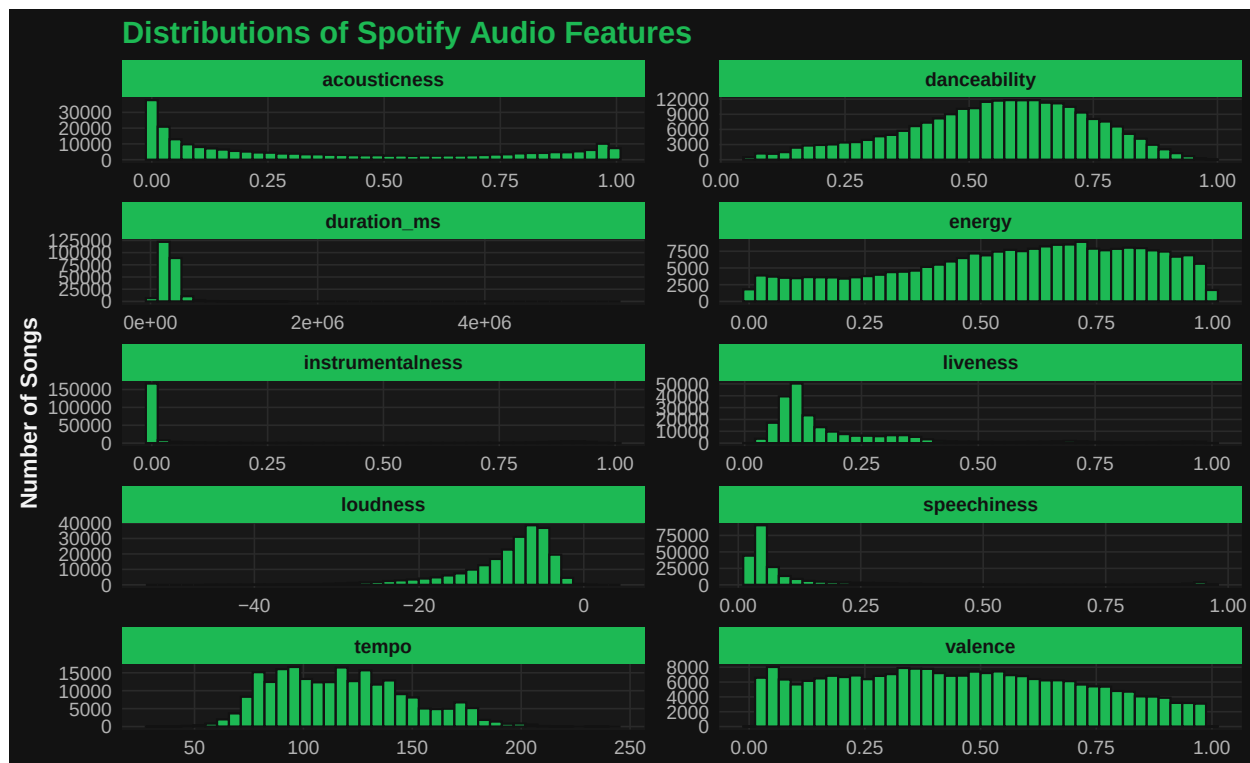
Table 8: Audio Feature Summary Statistics

feature	mean	median	sd
acousticness	0.369	0.232	0.355
danceability	0.554	0.571	0.186
duration_ms	235122.339	220427.000	118935.909
energy	0.571	0.605	0.263
instrumentalness	0.148	0.000	0.303
liveness	0.215	0.128	0.198
loudness	-9.570	-7.762	5.998
speechiness	0.121	0.050	0.186
tempo	117.667	115.778	30.899
valence	0.455	0.444	0.260

Audio Feature Distributions

Because the audio variables use different scales, faceting each variable with a free x-axis makes their shapes easier to compare.

```
spotify %>%
  select(all_of(audio_features)) %>%
  pivot_longer(everything(), names_to = "feature", values_to = "value") %>%
  ggplot(aes(x = value)) +
  geom_histogram(bins = 40, fill = "#1DB954", color = "#121212") +
  facet_wrap(~ feature, scales = "free", ncol = 2) +
  labs(
    title = "Distributions of Spotify Audio Features",
    x = NULL,
    y = "Number of Songs"
  ) +
  theme_spotify()
```



These distributions help identify skewness, extreme observations, and features whose typical values differ substantially from their ranges.

Audio Characteristics: Popular vs. Not Popular

```
group_feature_summary <- spotify_eda %>%
  group_by(popularity_group) %>%
  summarise(
    across(
      all_of(audio_features),
      ~mean(.x, na.rm = TRUE)
    ),
    .groups = "drop"
  ) %>%
  pivot_longer(
    cols = -popularity_group,
    names_to = "feature",
    values_to = "mean_value"
  )

knitr::kable(group_feature_summary, digits = 3,
  caption = "Mean Audio Characteristics by Popularity Group")
```

Table 9: Mean Audio Characteristics by Popularity Group

popularity_group	feature	mean_value
Not Popular	acousticness	0.483
Not Popular	danceability	0.507
Not Popular	duration_ms	238446.173
Not Popular	energy	0.518
Not Popular	instrumentalness	0.227
Not Popular	liveness	0.245
Not Popular	loudness	-11.539
Not Popular	speechiness	0.143
Not Popular	tempo	115.380
Not Popular	valence	0.438
Popular	acousticness	0.258
Popular	danceability	0.600
Popular	duration_ms	231926.943
Popular	energy	0.622
Popular	instrumentalness	0.072
Popular	liveness	0.186
Popular	loudness	-7.677
Popular	speechiness	0.099
Popular	tempo	119.865
Popular	valence	0.471

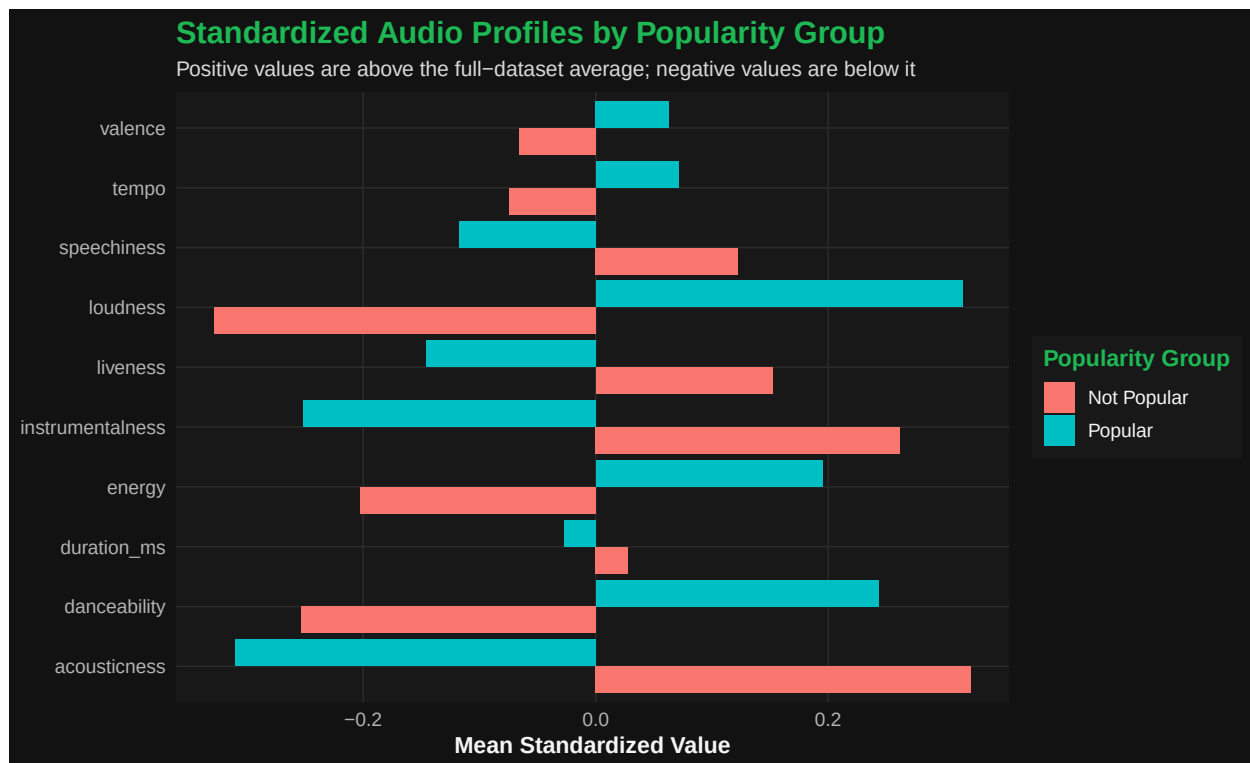
For clearer visual comparison, the following plot standardizes each audio characteristic before comparing popular and not-popular songs. Standardization is used here only for visualization; it does not alter the original data.

```

standardized_features <- spotify_eda %>%
  select(popularity_group, all_of(audio_features)) %>%
  mutate(across(all_of(audio_features), ~as.numeric(scale(.x)))) %>%
  pivot_longer(
    cols = all_of(audio_features),
    names_to = "feature",
    values_to = "standardized_value"
  ) %>%
  group_by(popularity_group, feature) %>%
  summarise(mean_standardized = mean(standardized_value, na.rm = TRUE), .groups = "drop")

ggplot(standardized_features,
  aes(x = feature, y = mean_standardized, fill = popularity_group)) +
  geom_col(position = "dodge") +
  coord_flip() +
  labs(
    title = "Standardized Audio Profiles by Popularity Group",
    subtitle = "Positive values are above the full-dataset average; negative values are below it",
    x = NULL,
    y = "Mean Standardized Value",
    fill = "Popularity Group"
  ) +
  theme_spotify()

```

This plot highlights which audio characteristics differ most between the two popularity groups while putting all variables on a common visual scale.

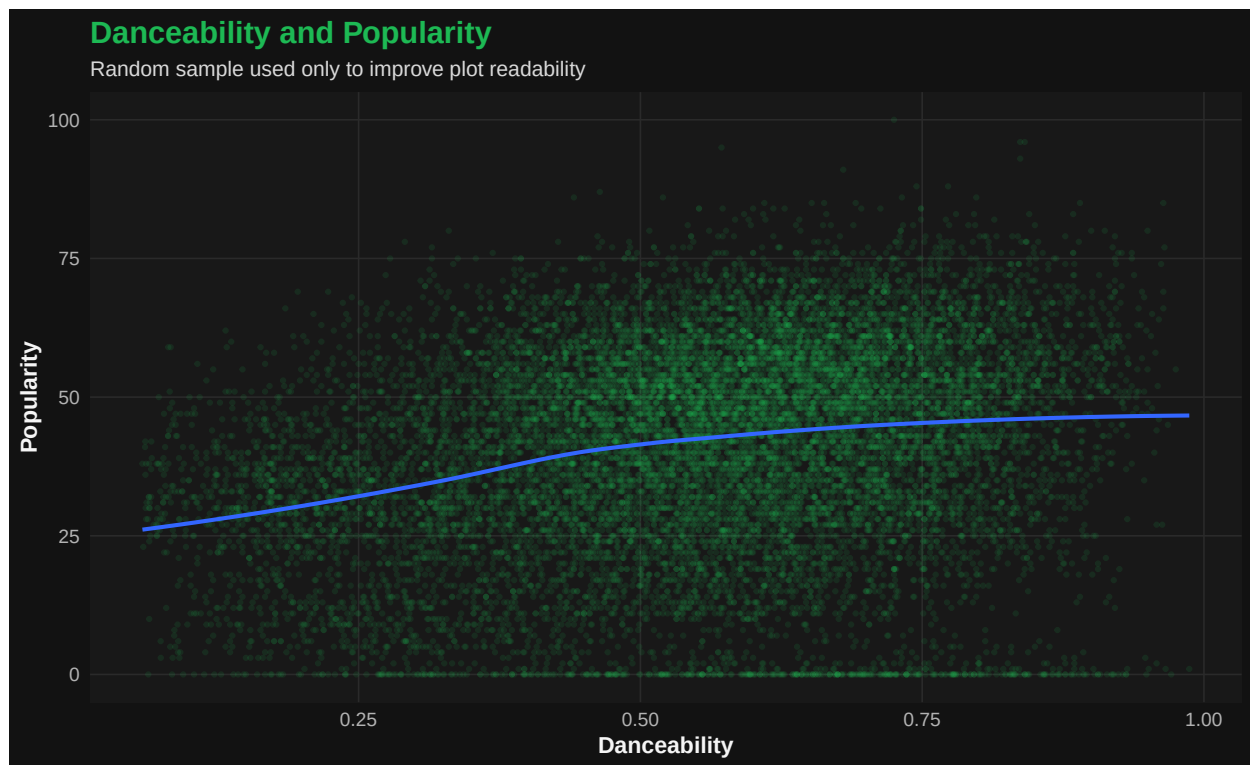
Selected Feature Relationships with Popularity

To make scatterplots readable with more than 230,000 observations, a reproducible random sample is used only for visualization. Summary statistics elsewhere in this report continue to use the complete dataset.

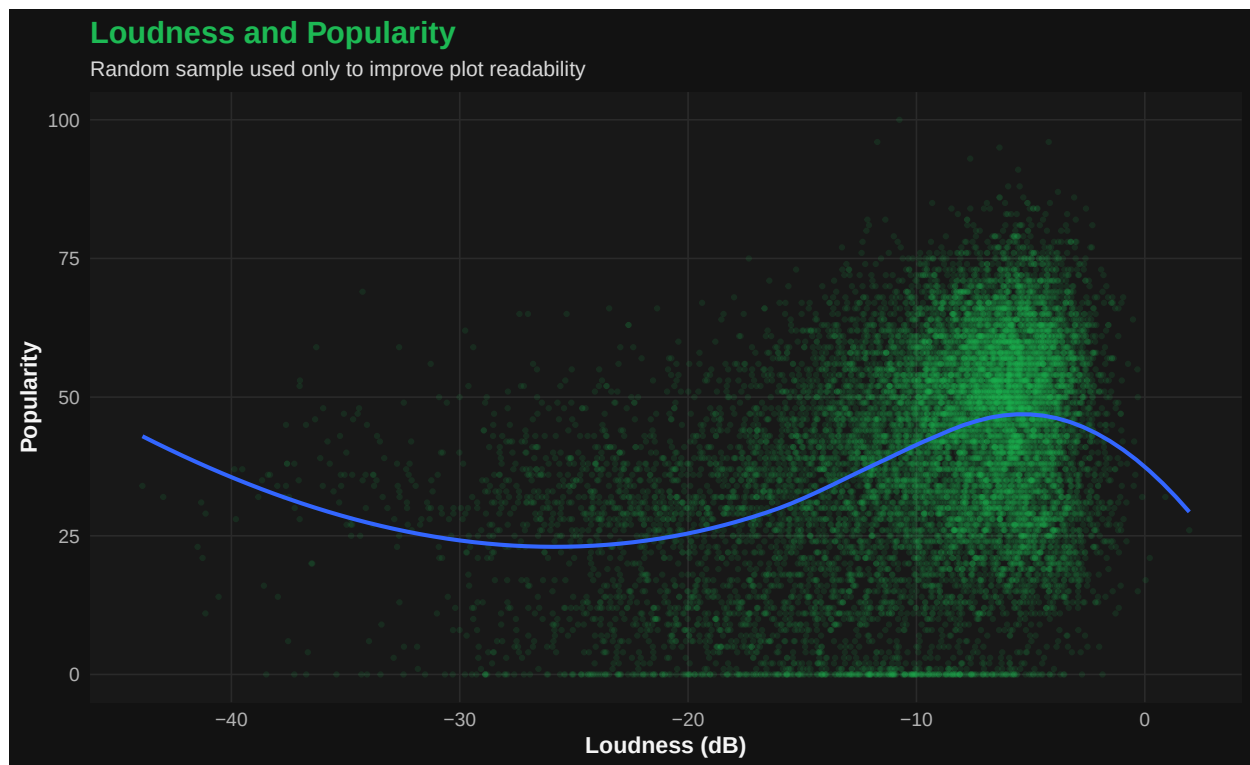
```
set.seed(12345)

spotify_plot_sample <- spotify %>%
  slice_sample(n = min(15000, nrow(spotify)))

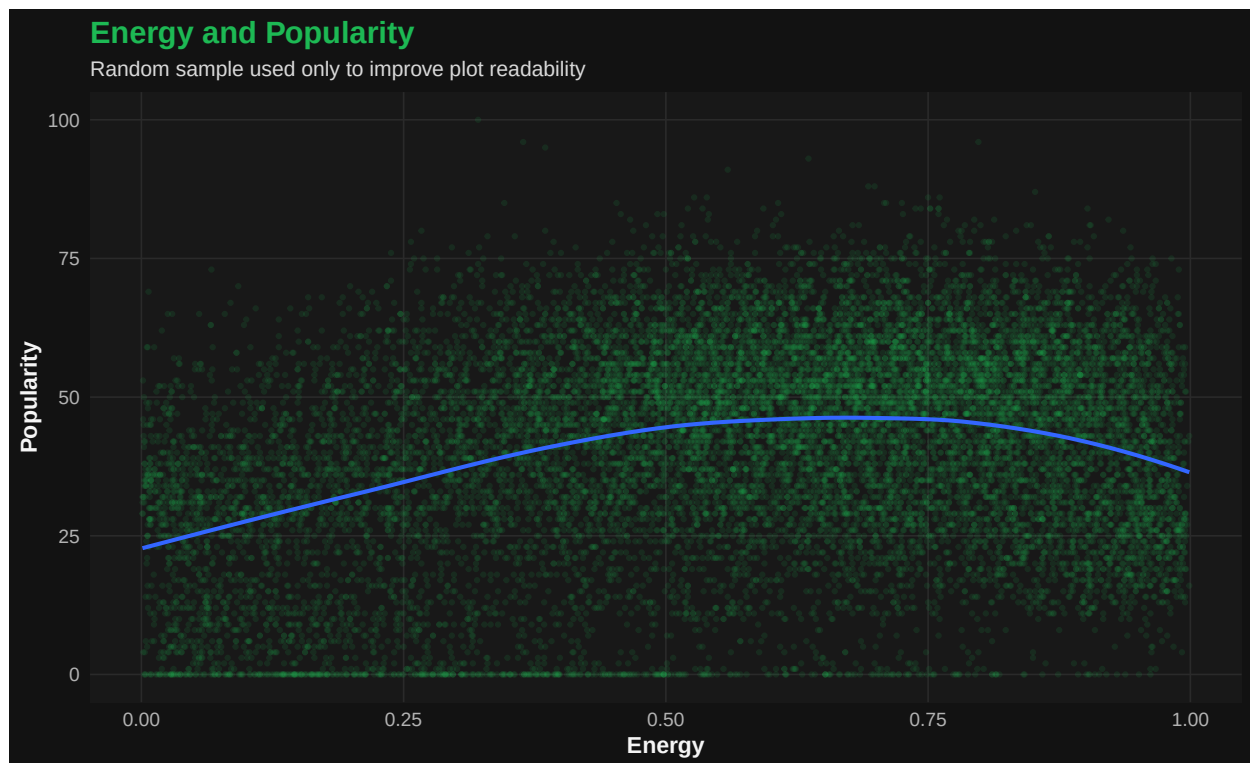
ggplot(spotify_plot_sample, aes(x = danceability, y = popularity)) +
  geom_point(alpha = 0.12, size = 0.8, color = "#1DB954") +
  geom_smooth(method = "loess", se = FALSE, linewidth = 1) +
  labs(
    title = "Danceability and Popularity",
    subtitle = "Random sample used only to improve plot readability",
    x = "Danceability",
    y = "Popularity"
  ) +
  theme_spotify()
```



```
ggplot(spotify_plot_sample, aes(x = loudness, y = popularity)) +  
  geom_point(alpha = 0.12, size = 0.8, color = "#1DB954") +  
  geom_smooth(method = "loess", se = FALSE, linewidth = 1) +  
  labs(  
    title = "Loudness and Popularity",  
    subtitle = "Random sample used only to improve plot readability",  
    x = "Loudness (dB)",  
    y = "Popularity"  
  ) +  
  theme_spotify()
```



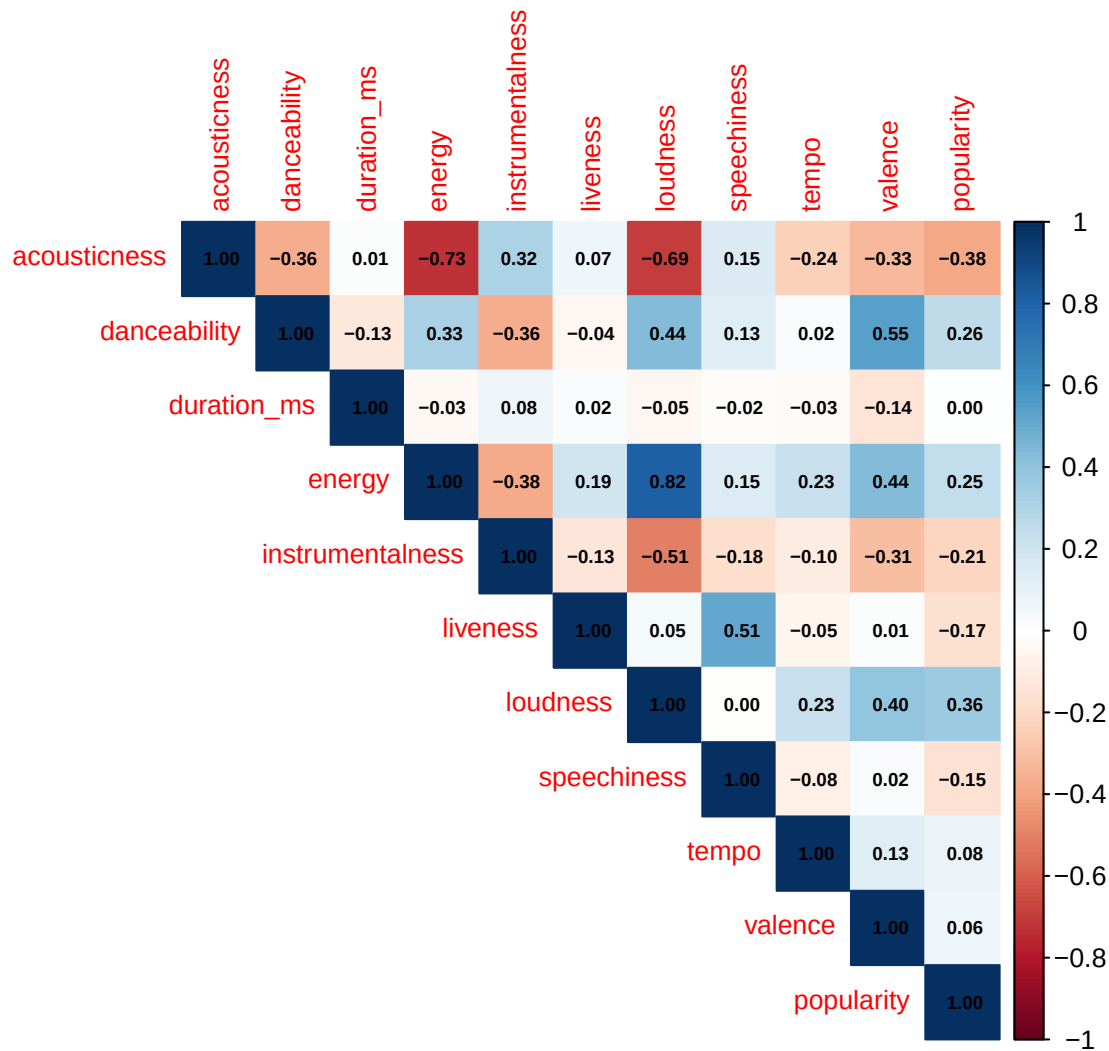
```
ggplot(spotify_plot_sample, aes(x = energy, y = popularity)) +  
  geom_point(alpha = 0.12, size = 0.8, color = "#1DB954") +  
  geom_smooth(method = "loess", se = FALSE, linewidth = 1) +  
  labs(  
    title = "Energy and Popularity",  
    subtitle = "Random sample used only to improve plot readability",  
    x = "Energy",  
    y = "Popularity"  
  ) +  
  theme_spotify()
```



The smoothed trend lines are descriptive rather than causal. They show the direction and shape of association but do not establish that changing an audio characteristic would cause a change in popularity.

Correlation Analysis

```
numeric_features <- spotify %>%  
  select(  
    acousticness, danceability, duration_ms, energy,  
    instrumentalness, liveness, loudness, speechiness,  
    tempo, valence, popularity  
  )  
  
cor_matrix <- cor(numeric_features, use = "complete.obs")  
  
corrplot(  
  cor_matrix,  
  method = "color",  
  type = "upper",  
  tl.cex = 0.8,  
  addCoef.col = "black",  
  number.cex = 0.55  
)
```



```
popularity_correlations <- data.frame(
  feature = rownames(cor_matrix),
  correlation_with_popularity = cor_matrix[, "popularity"]
) %>%
  filter(feature != "popularity") %>%
  arrange(desc(abs(correlation_with_popularity)))

knitr::kable(popularity_correlations, digits = 3,
  caption = "Correlations with Popularity")
```

Table 10: Correlations with Popularity

	feature	correlation_with_popularity
acousticness	acousticness	-0.381
loudness	loudness	0.363
danceability	danceability	0.257
energy	energy	0.249
instrumentalness	instrumentalness	-0.211
liveness	liveness	-0.168

	feature	correlation_with_popularity
speechiness	speechiness	-0.151
tempo	tempo	0.081
valence	valence	0.060
duration_ms	duration_ms	0.002

Correlation measures linear association. A low correlation does not necessarily mean a feature has no predictive value, particularly when relationships are nonlinear or interact with genre.

Genre-Level Audio Profiles

```
genre_feature_summary <- spotify %>%
  group_by(genre) %>%
  summarise(
    danceability = mean(danceability, na.rm = TRUE),
    energy = mean(energy, na.rm = TRUE),
    valence = mean(valence, na.rm = TRUE),
    acousticness = mean(acousticness, na.rm = TRUE),
    loudness = mean(loudness, na.rm = TRUE),
    tempo = mean(tempo, na.rm = TRUE),
    popularity = mean(popularity, na.rm = TRUE),
    count = n(),
    .groups = "drop"
  )

knitr::kable(genre_feature_summary, digits = 2,
  caption = "Genre-Level Audio and Popularity Summary")
```

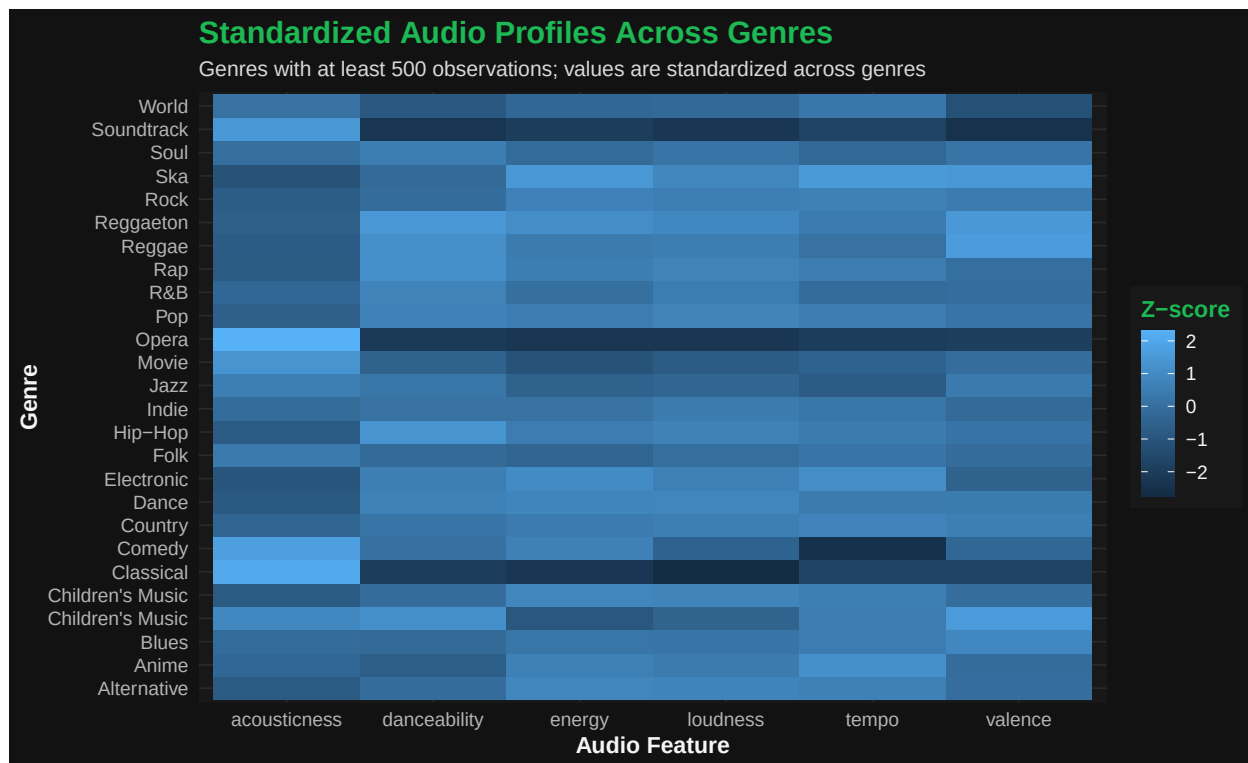
Table 11: Genre-Level Audio and Popularity Summary

genre	danceability	energy	valence	acousticness	loudness	tempo	popularity	count
A Capella	0.41	0.25	0.33	0.83	-13.66	111.52	9.30	119
Alternative	0.54	0.71	0.45	0.16	-6.54	122.53	50.21	9263
Anime	0.47	0.67	0.44	0.29	-7.92	126.63	24.26	8936
Blues	0.53	0.61	0.58	0.33	-9.05	121.14	34.74	9023
Children's Music	0.70	0.40	0.68	0.59	-11.64	121.13	4.25	5403
Children's Music	0.54	0.71	0.45	0.16	-6.53	121.93	54.66	9353
Classical	0.31	0.18	0.21	0.87	-21.54	104.34	29.28	9256
Comedy	0.56	0.68	0.41	0.79	-11.69	98.24	21.34	9681
Country	0.58	0.64	0.54	0.27	-7.34	123.41	46.10	8664
Dance	0.64	0.70	0.52	0.15	-6.05	120.80	57.28	8701
Electronic	0.62	0.74	0.39	0.12	-7.04	125.85	38.06	9377
Folk	0.53	0.49	0.44	0.46	-9.87	118.75	49.94	9299
Hip-Hop	0.72	0.64	0.47	0.18	-6.86	120.79	58.42	9295
Indie	0.57	0.58	0.43	0.33	-7.92	119.29	54.70	9543
Jazz	0.59	0.47	0.51	0.50	-11.21	111.78	40.82	9441

genre	danceability	energy	valence	acousticness	loudness	tempo	popularity	count
Movie	0.49	0.37	0.45	0.70	-12.95	113.95	12.17	7806
Opera	0.29	0.17	0.19	0.95	-19.34	101.80	13.34	8280
Pop	0.64	0.64	0.48	0.22	-6.50	121.18	66.59	9386
R&B	0.64	0.56	0.45	0.29	-7.60	116.37	52.31	8992
Rap	0.70	0.65	0.46	0.17	-6.67	121.10	60.53	9232
Reggae	0.70	0.64	0.68	0.19	-7.52	118.16	35.59	8771
Reggaeton	0.73	0.75	0.66	0.22	-5.88	120.99	37.74	8927
Rock	0.54	0.68	0.52	0.20	-7.29	122.63	59.62	9272
Ska	0.53	0.82	0.65	0.10	-6.17	129.43	28.61	8874
Soul	0.62	0.53	0.48	0.36	-8.87	115.32	47.03	9089
Soundtrack	0.27	0.22	0.12	0.72	-19.28	104.08	33.95	9646
World	0.44	0.51	0.30	0.39	-10.71	119.82	35.52	9096

```
genre_heatmap <- genre_feature_summary %>%
  filter(count >= 500) %>%
  select(genre, danceability, energy, valence, acousticness, loudness, tempo) %>%
  mutate(across(-genre, ~as.numeric(scale(.x)))) %>%
  pivot_longer(-genre, names_to = "feature", values_to = "z_score")

ggplot(genre_heatmap, aes(x = feature, y = genre, fill = z_score)) +
  geom_tile() +
  labs(
    title = "Standardized Audio Profiles Across Genres",
    subtitle = "Genres with at least 500 observations; values are standardized across genres",
    x = "Audio Feature",
    y = "Genre",
    fill = "Z-score"
  ) +
  theme_spotify()
```



The heatmap makes it easier to see how genres differ simultaneously across multiple musical characteristics rather than comparing one feature at a time.

Popularity Variation Within Genre

A genre can have a high average popularity while still containing substantial variation among its tracks. The coefficient of variation is not ideal for a bounded variable such as popularity, so this section instead compares the interquartile range (IQR) of popularity across genres.

```
genre_variation <- spotify %>%
  group_by(genre) %>%
  summarise(
    median_popularity = median(popularity),
    iqr_popularity = IQR(popularity),
    min_popularity = min(popularity),
    max_popularity = max(popularity),
    n = n(),
    .groups = "drop"
  ) %>%
  arrange(desc(iqr_popularity))

knitr::kable(genre_variation, digits = 2,
  caption = "Within-Genre Variation in Popularity")
```


Table 12: Within-Genre Variation in Popularity

genre	median_popularity	iqr_popularity	min_popularity	max_popularity	n
Movie	7	19	0	82	7806
Reggaeton	35	18	0	98	8927
Reggae	34	14	0	78	8771
Ska	27	14	5	74	8874
Anime	23	13	0	65	8936
Classical	32	13	0	69	9256
Country	45	13	0	82	8664
Dance	57	13	0	100	8701
Electronic	37	13	0	96	9377
Blues	33	12	0	80	9023
R&B	51	12	0	92	8992
Soul	46	12	0	85	9089
World	34	12	0	76	9096
Comedy	20	11	0	61	9681
Folk	49	11	0	84	9299
Hip-Hop	57	11	14	98	9295
Jazz	40	11	0	79	9441
Soundtrack	33	11	0	72	9646
Alternative	49	10	0	83	9263
Indie	54	10	0	97	9543
Opera	11	10	0	63	8280
Rap	59	10	14	99	9232
Rock	59	10	0	95	9272
A Capella	8	9	0	44	119
Children’s Music	54	9	0	86	9353
Pop	66	9	3	100	9386
Children’s Music	0	4	0	51	5403

This table helps distinguish genres with consistently similar popularity from genres that contain both very low- and very high-popularity songs.

Key Findings

The exploratory analysis should be interpreted as descriptive evidence rather than causal evidence. The most important conclusions to carry into any later modeling are:

1. The popularity classes created using the median threshold are close to evenly balanced.
2. Genre is likely to be an important explanatory variable because average popularity and audio characteristics vary substantially across genres.
3. Popular and not-popular songs differ across several measurable audio characteristics, although no single feature appears sufficient to explain popularity by itself.
4. Some audio features are strongly related to one another, so models that use several predictors simultaneously may capture information not visible in one-variable comparisons.
5. Relationships between audio characteristics and popularity should be interpreted as associations. The dataset alone does not establish that changing a song characteristic causes higher popularity.
6. Repeated track IDs should be considered when interpreting song-level counts, because the dataset contains fewer unique track IDs than total rows.